

Micromouse Engineering Specification & Implementation Manual

MCU: ATmega328P (5V, 16MHz)

Maze Pitch: 180 mm (16×16)

IMU: ICM-20602 (I2C Fast)

SRAM Budget: 1,320 / 2,048 B

1. HARDWARE PIN ASSIGNMENTS & ELECTRICAL CONTRACTS

Subsystem	Pin	I/O Type	Electrical Contract & Firmware Constraints
Left Motor	D5 , D7 , D8	PWM, DO, DO	D5 : Timer0B PWM (PWM_A). D7/D8 : AIN1/AIN2 . Brake: both HIGH.
Right Motor	D6 , D9 , D10	PWM, DO, DO	D6 : Timer0A PWM (PWM_B). D9/D10 : BIN1/BIN2 . STBY pin tied hard to 5V.
Left Encoder	D2 , D4	INT0, DI	D2 : External Interrupt RISING. D4 : Polled in ISR for quadrature direction.
Right Encoder	D3 , D11	INT1, DI	D3 : External Interrupt RISING. D11 : Polled in ISR for quadrature direction.
ICM-20602 IMU	A4 , A5	I2C (SDA/SCL)	Addr 0x68 . Set Wire.setClock(400000) . Direct register reads; avoid heavy libs.
Bluetooth	D12 , D13	SoftRX, SoftTX	SoftwareSerial 9600 baud. D13 output requires 1k/2k divider to 3.3V RXD.
Sharp IR x4	A0 – A3	Analog In	Dual front on A3/A0 ; 45° angled on A1/A2 . 10-bit ADC (0–1023), 5-tap filter.

2. PRODUCTION HEADER DEFINITIONS (`CONFIG.H`)

```
#pragma once
#include <Arduino.h>

// --- KINEMATICS & TRANSLATION ---
#define BASE_SPEED      80      // Calibrated cruise PWM
#define MIN_DRIVE_SPEED  40      // Anti-stall floor during deceleration
#define TICKS_PER_CELL   600L   // 180mm travel (33.33 ticks/cm, 3.33 ticks/mm)

// --- GYRO ROTATION CONTROLLER ---
#define ICM_ADDR         0x68
#define GYRO_CONFIG_VAL  0x10    // FS = ±1000 dps (prevents pivot saturation)
#define GYRO_SCALE       32.18f  // Calibrated scale factor: (raw - offset) / 32.18
#define GYRO_DEADBAND    0.80f   // Zero-rate drift elimination threshold (dps)
#define BRAKE_LEAD_DEG   5.00f   // Pre-stop lead angle absorbing rotor/chassis inertia

// --- SENSOR CLEARANCE CALIBRATION (Clearance mm = (raw - offset) / slope) ---
#define IR_AL_OFFSET     280.0f   // A1: Angled Left
#define IR_AL_SLOPE      8.98f
#define IR_AR_OFFSET     338.0f   // A2: Angled Right
#define IR_AR_SLOPE      10.28f
#define IR_FL_OFFSET     396.0f   // A3: Front Left
#define IR_FL_SLOPE      9.20f
#define IR_FR_OFFSET     383.0f   // A0: Front Right
#define IR_FR_SLOPE      9.05f

// --- OPERATIONAL THRESHOLDS ---
#define SIDE_WALL_DETECT_MM 80.0f // Clearance < 80mm confirms diagonal wall exists
#define NOMINAL_SIDE_SET_MM 45.0f // Ideal tracking distance in corridor
#define FRONT_WALL_DETECT_MM 70.0f // Confirms front wall presence in next node
#define FRONT_EMERGENCY_STOP 35.0f // Chassis collision abort ceiling
```

3. HARDWARE DRIVERS & CALIBRATED MATH IMPLEMENTATION

```
// Direct Register Read: Gyro Z-axis
int16_t readGyroZRaw() {
    Wire.beginTransmission(ICM_ADDR);
    Wire.write(0x47); // GYRO_ZOUT_H register
    Wire.endTransmission(false);
    Wire.requestFrom((uint8_t)ICM_ADDR, (uint8_t)2);
    return (Wire.read() << 8) | Wire.read();
}

// Non-blocking Integration with Deadband
void updateGyro(float &angleZ, unsigned long &lastMicros, float offset) {
    unsigned long now = micros();
    float dt = (now - lastMicros) * 0.000001f;
    lastMicros = now;
```

```
float dps = (readGyroZRaw() - offset) / GYRO_SCALE;
if (fabs(dps) > GYRO_DEADBAND) {
    angleZ += dps * dt;
}

// Convert Sensor ADC to Physical Millimeters
inline float getClearanceMM(float raw, float offset, float slope) {
    float mm = (raw - offset) / slope;
    return (mm < 0.0f) ? 0.0f : mm;
}
```

4. CORE MOVEMENT PRIMITIVES

Calibrated Pivot-in-Place (`rotatePrecise`)

```
void rotatePrecise(float targetAngle, bool turnRight, float gyroOffset) {
    float currentAngle = 0.0f;
    unsigned long lastMicros = micros();
    unsigned long startMillis = millis();

    digitalWrite(AIN1, turnRight ? HIGH : LOW);
    digitalWrite(AIN2, turnRight ? LOW : HIGH);
    digitalWrite(BIN1, turnRight ? LOW : HIGH);
    digitalWrite(BIN2, turnRight ? HIGH : LOW);

    // Cut off power early to let chassis coast flush to target
    float cutoffTarget = targetAngle - BRAKE_LEAD_DEG;

    while (fabs(currentAngle) < cutoffTarget) {
        updateGyro(currentAngle, lastMicros, gyroOffset);
        if (millis() - startMillis > 2000) break; // Safety timeout

        float remaining = targetAngle - fabs(currentAngle);
        int spd = (remaining < 30.0f) ?
            map((int)remaining, 0, 30, MIN_DRIVE_SPEED, BASE_SPEED) : BASE_SPEED;
        analogWrite(PWMA, spd);
        analogWrite(PWMB, spd);
    }
    brakeMotors();
    delay(80); // Settle momentum
}
```

Cell Step Translation with Closed-Loop Centering (`stepForwardOneCell`)

```

void stepForwardOneCell(float Kp, float Kd) {
    noInterrupts(); leftTicks = 0; rightTicks = 0; interrupts();
    digitalWrite(AIN1, HIGH); digitalWrite(AIN2, LOW);
    digitalWrite(BIN1, HIGH); digitalWrite(BIN2, LOW);

    bool lDone = false, rDone = false;
    float prevError = 0.0f;

    while (!lDone || !rDone) {
        noInterrupts(); long cL = labs(leftTicks); long cR = labs(rightTicks); interrupts();

        float fL = getClearanceMM(sensorFL.getDist(), IR_FL_OFFSET, IR_FL_SLOPE);
        float fR = getClearanceMM(sensorFR.getDist(), IR_FR_OFFSET, IR_FR_SLOPE);
        if (((fL + fR) * 0.5f) < FRONT_EMERGENCY_STOP) break;

        if (cL >= TICKS_PER_CELL) lDone = true;
        if (cR >= TICKS_PER_CELL) rDone = true;

        float aL = getClearanceMM(sensorAL.getDist(), IR_AL_OFFSET, IR_AL_SLOPE);
        float aR = getClearanceMM(sensorAR.getDist(), IR_AR_OFFSET, IR_AR_SLOPE);
        bool hasL = (aL < SIDE_WALL_DETECT_MM);
        bool hasR = (aR < SIDE_WALL_DETECT_MM);

        float error = 0.0f;
        if (hasL && hasR) error = aL - aR;
        else if (hasL) error = (aL - NOMINAL_SIDE_SET_MM) * 1.4f;
        else if (hasR) error = (NOMINAL_SIDE_SET_MM - aR) * 1.4f;
        else error = (cL - cR) * 0.4f; // Encoder sync fallback

        float steer = (error * Kp) + ((error - prevError) * Kd);
        prevError = error;

        int lPwm = constrain(BASE_SPEED - (int)steer, 0, 255);
        int rPwm = constrain(BASE_SPEED + (int)steer, 0, 255);

        analogWrite(PWMA, lDone ? 0 : lPwm);
        analogWrite(PWMB, rDone ? 0 : rPwm);
    }
    brakeMotors();
    delay(60);
}

```

5. 16×16 MEMORY-OPTIMIZED MAZE ARCHITECTURE

SRAM Budget Target: ATmega328P has 2,048 bytes of SRAM. Traditional 2-byte coordinate structs allocate 512 bytes for a full BFS queue, triggering stack corruption. Nibble encoding compresses `(x, y)` into an 8-bit `uint8\_t`, cutting queue RAM to exactly 256 bytes.

Structure	Memory Size	Encoding Schema & Bit Layout
distances[16][16]	256 Bytes	uint8_t . Distance potential to center goals (0 to 254). 255 = unvisited.
walls[16][16]	256 Bytes	Bitmask: North = 0x01, East = 0x02, South = 0x04, West = 0x08 .
floodQueue[256]	256 Bytes	Packed Byte: High Nibble (bits 7:4) = Y, Low Nibble (bits 3:0) = X. Circular pointer.
Total Static Grid	768 Bytes	Leaves 1,280 Bytes free for stack frames, interrupt ISRs, and Serial buffers.

```

// Nibble Encoding Macros
#define PACK_COORD(x, y) (((uint8_t)(y) << 4) | ((uint8_t)(x) & 0x0F))
#define UNPACK_X(packed) ((uint8_t)((packed) & 0x0F))
#define UNPACK_Y(packed) ((uint8_t)((packed) >> 4))

// Push / Pop Circular Ring Buffer (Index auto-wraps on uint8_t overflow)
uint8_t floodQueue[256];
uint8_t qHead = 0;
uint8_t qTail = 0;

inline void enqueueCell(uint8_t x, uint8_t y) {
    floodQueue[qTail++] = PACK_COORD(x, y);
}

inline void dequeueCell(uint8_t &x, uint8_t &y) {

```

```
uint8_t packed = floodQueue[qHead++];  
x = UNPACK_X(packed);  
y = UNPACK_Y(packed);  
}
```

Implementation Rule: When setting walls in `walls[x][y]`, always set the corresponding opposite wall in the adjacent cell (e.g., if cell (x, y) has an East wall, cell $(x+1, y)$ must have a West wall) to ensure bidirectional consistency in the BFS propagation.